

PULSE**RAISE**

SECURITY
AUDIT REPORT

AUDITED BY

ZUHAIBMOHD



Pulse Raise Security Review Report

Introduction

A security review of the Pulse Raise crowdfunding program was conducted by **zuhaibmohd**, with a focus on the security aspects of the Solana smart contracts.

Disclaimer

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource, and expertise-bound effort where I attempt to find as many vulnerabilities as possible.

⚠ Note

We cannot guarantee 100% security after the review or even that the review will uncover any vulnerabilities.

It is recommended to conduct subsequent reviews, run bug bounty programs, and enable on-chain monitoring to ensure long-term security.

About zuhaibmohd

zuhaibmohd is an independent smart contract security researcher.

Check out his [previous work](#) or connect on X [@zuhaib44](#).

About Pulse Raise

Pulse Raise is an **Anchor program** on Solana that manages crowdfunding campaigns with instructions to initialize protocol configuration, create campaigns, accept donations, update campaign metadata, update protocol fee settings, withdraw funds, and deactivate campaigns.

Severity Classification

Severity	Likelihood ->	High	Medium	Low
Impact v				
High	Critical	High	Medium	
Medium	High	Medium	Low	
Low	Medium	Low	Low	Low

- **Impact:** The potential technical, economic, and reputational damage from a successful exploit.
- **Likelihood:** The probability that a given vulnerability will be discovered and exploited.
- **Severity:** The overall criticality based on the above two factors.

Informational

Findings in this category are recommended improvements to enhance code structure, usability, and overall system effectiveness.

Scope

The following was in scope for this audit:

- **Program:** Pulse Raise (Anchor program)
- **Codebase:** `anchor-contract/programs/pulse-raise/` (Rust/Anchor)

Findings Summary

Severity	Count
High	1
Medium	1
Low	1
Informational	0

Total Severity	Count
-------------------	-------

Detailed Findings

[HIGH] Arbitrary CPI Target in Donation Flow (Unchecked Program ID)

ID: PROJECT-001
Contract / File: anchor-contract/programs/pulse-raise/src/instructions/donate.rs
Lines: L38-L39, L102-L132
Severity: High

Summary

The donation instruction accepts `jito_stake_pool_program` as an `UncheckedAccount` and uses it directly as the CPI target for `deposit_sol`. Since the program ID is not pinned, a caller can provide an arbitrary executable program that returns success without real stake-pool semantics. This matters because campaign accounting (`donations`, `amt_raised`) is updated after CPI success, allowing state corruption.

Root Cause

`jito_stake_pool_program` is declared as an `UncheckedAccount` and passed directly into CPI construction without an address constraint or explicit key check against the expected stake-pool program ID.

Impact

An attacker can falsify fundraising progress by forcing logically invalid donations to be treated as successful.

Attack Scenario

1. A campaign is active and users can call `lets_donate`.
2. The attacker submits a malicious executable account as `jito_stake_pool_program`.
3. The program performs CPI against the attacker-selected program, which returns success.
4. The contract increments donation counters despite missing intended external behavior.
5. The campaign reaches fake milestones and reports incorrect raised amounts.

Recommendation

Pin the external program ID using account constraints and fail closed on mismatch. Add a runtime key equality check as defense-in-depth before constructing CPI instruction data.

Place the check in `Donation` account validation and immediately before the `deposit_sol` call.

Code Snippet

```
// Vulnerable code
/// CHECK: stake pool program id
pub jito_stake_pool_program: UncheckedAccount<'info>,

let ix = deposit_sol(
    &self.jito_stake_pool_program.key(),
    // ...
    amount,
);
invoke(&ix, &[/ * ... */]);
self.campaign.amt_raised = self.campaign.amt_raised.saturating_add(amount);

// Example fix
/// CHECK: validated by address constraint
#[account(address = spl_stake_pool::id())]
pub jito_stake_pool_program: UncheckedAccount<'info>,

require_keys_eq!(
    self.jito_stake_pool_program.key(),
    spl_stake_pool::id(),
    ErrorCode::InvalidProgramId
);
```

[MEDIUM] Campaign Can Be Mutated or Deactivated Mid-Lifecycle (Trust/Integrity Risk)

ID: PROJECT-003

Contract / File: anchor-contract/programs/pulse-raise/src/instructions/update_campaign.rs, anchor-contract/programs/pulse-raise/src/instructions/delete_campaign.rs, anchor-contract/programs/pulse-raise/src/instructions/create_campaign.rs

Lines: update_campaign.rs L32-L40, delete_campaign.rs L25-L27

Severity: Medium

Summary

Campaign creators can edit key metadata and goals at any time and can also deactivate campaigns without lifecycle boundaries. There is no state machine enforcing pre-start vs live vs ended phases. This matters because donors can be exposed to post-donation goal changes or abrupt shutdowns without predictable policy.

Root Cause

The campaign state model only relies on a boolean active flag and lacks lifecycle timestamps/status checks that constrain when mutable operations are allowed.

Impact

A malicious creator can manipulate campaign terms after collecting momentum, undermining integrity and donor trust.

Worst case: protocol invariant break where campaign promises are changed while still appearing valid in protocol state.

Attack Scenario

1. Donors begin funding a campaign based on initial metadata and goal.
2. Creator updates title/description/goal while campaign is active.
3. Creator optionally deactivates campaign abruptly.
4. Protocol accepts all changes because no lifecycle lock exists.
5. Donors are left with altered campaign state and no on-chain guarantees.

Recommendation

Introduce lifecycle fields (`start_time`, `end_time`, optional `status`) and enforce phase-based permissions. Freeze sensitive fields once campaign is live and permit only narrowly scoped operations after close.

Place checks in `updating_campaign` and `deleting_campaign`, and initialize lifecycle fields during creation.

Code Snippet

```
// Vulnerable code
self.campaign.title = title;
self.campaign.desc = desc;
self.campaign.image_url = img_uri;
self.campaign.goal = goal;

require!(self.campaign.active, ErrorCode::InactiveCampaign);
self.campaign.active = false;

// Example fix
require!(
    Clock::get()?.unix_timestamp < self.campaign.start_time,
    ErrorCode::CampaignLiveImmutable
);
// allow only non-sensitive metadata before start_time
```

[LOW] Saturating Arithmetic Silently Masks Accounting Overflow Conditions

ID: PROJECT-007

Contract / File: anchor-contract/programs/pulse-raise/src/instructions/donate.rs, anchor-contract/programs/pulse-raise/src/instructions/create_campaign.rs, anchor-contract/programs/pulse-raise/src/instructions/withdraw.rs

Lines: donate.rs L131-L132, create_campaign.rs L73, withdraw.rs L146

Severity: Low

Summary

The contract uses `saturating_add` for core counters and accounting values. Saturation prevents hard failure and instead clamps values at `u64::MAX`, which can conceal invariant violations. This matters in financial logic because silent overflow handling obscures data integrity problems.

Root Cause

Financial counters are updated with `saturating_add` rather than checked arithmetic, so overflow states are masked instead of being surfaced and reverted.

Impact

Attackers or edge cases can push values toward overflow and leave the protocol in a state that appears valid while being mathematically incorrect.

Worst case: protocol invariant break where accounting is permanently inaccurate without explicit failure signal.

Attack Scenario

1. A hot campaign receives repeated updates over long periods.
2. Counter fields approach numeric limits.
3. Arithmetic clamps instead of reverting on overflow edge.
4. Protocol continues operating with silently altered values.
5. Monitoring and users read corrupted but non-failing state.

Recommendation

Use `checked_add` and return explicit overflow errors to fail fast. Apply the same policy to all financial counters and totals for consistent invariant enforcement.

Place replacements exactly where `saturating_add` is currently used in donation, campaign creation, and withdrawal counters.

Code Snippet

```
// Vulnerable code
self.campaign.donations = self.campaign.donations.saturating_add(1);
self.campaign.amt_raised = self.campaign.amt_raised.saturating_add(amount);
self.program_config.campaign_count = self.program_config.campaign_count.saturating_add(1);

// Example fix
self.campaign.donations = self
    .campaign
    .donations
    .checked_add(1)
    .ok_or(ErrorCode::MathOverflow)?;
self.campaign.amt_raised = self
    .campaign
    .amt_raised
    .checked_add(amount)
    .ok_or(ErrorCode::MathOverflow)?;
```